Практическая работа №4 — Теоретический конспект

Зачем MDA: исторические предпосылки и мотивация: К концу 90-х развилось множество несовместимых технологий для распределённых систем: COM/DCOM, CORBA, Java/RMI, XML/SOAP и др. Разработчикам приходилось «вшивать» платформенные детали прямо в архитектуру, из-за чего страдали переносимость, интеграция и эволюция систем. MDA появилась как ответ: отделить **суть системы** (модель) от **способа её реализации** (платформа), чтобы снизить риски от технологических различий и облегчить миграции между стеками.

OMG сознательно ушла от идеи «единого стандарта интеграции» к **интеграции стандартов**: MDA — это не «одна технология», а общая рамка, позволяющая совместно использовать разные стандарты и инструменты, сохраняя инвестиции в бизнес-логику при смене платформ.

Что такое MDA

MDA — модельно-ориентированный подход, где вы строите **платформенно-независимую модель (PIM)**, а затем **трансформируете** её в одну или несколько **платформенно-зависимых моделей (PSM)** и, при наличии инструментов, генерируете артефакты (код, схемы БД, контракты). OMG задаёт стандарты и принципы, а конкретные средства трансформации и генерации предоставляют сторонние разработчики.

Ядро стандартов MDA

- **UML** — унифицированный язык описания моделей (структура и поведение).
- **MOF** — мета-метамодель: общий «язык» описания метамоделей (на нём формально описаны метамодели UML и CWM).
- **CWM** — набор метамоделей для хранилищ данных и аналитики (реляционные, OLAP, ETL и пр.).
- **XMI** — обмен моделями/метаданными в XML; даёт «самоописываемые» документы: метамодель + модель в одном артефакте.

Практический пример CWM: различные движки (СУБД, OLAP, ETL) берут из общей модели **только свой «срез»** метаданных, что позволяет сквозную генерацию/согласование на уровне одного источника правды.

Три уровня моделей: CIM → PIM → PSM

- **CIM (Computation-Independent Model)** — контекст/предметная область, акторы, цели, ограничения. Максимум бизнес-смысла, минимум техники.
- **PIM (Platform-Independent Model)** — **состав, структура и поведение** системы без деталей реализации. Строится на основе CIM, чаще на UML; допускает формализацию поведения (Action Semantics) и инвариантов (например, OCL).
- **PSM (Platform-Specific Model)** — конкретизация под платформу (языки, протоколы, СУБД, развертывание). Одна PIM может порождать **несколько PSM**, а между ними возможно генерировать **bridges** — связки для согласования.

Трансформации и генерация

Идея MDA — **явные правила** перехода PIM→PSM (таблицы соответствий и/или формальные трансформации) и **воспроизводимая генерация** артефактов. OMG задаёт стандарты и требования к моделям, но не поставляет «волшебной кнопки»; инструментальную поддержку реализуют сторонние экосистемы.

Типичные соответствия (пример):

- Сущность PIM → Таблица/Коллекция в PSM (PK/FK/индексы из инвариантов).
- Ассоциация → Ограничения целостности, внешние ключи, события домена.
- Сервис/Операция → REST/gRPC-контракт (OpenAPI/Proto), семантика идемпотентности.
- Событие → Топик/Очередь + схема события.
- Ограничение/Инвариант → CHECK/уникальные ключи/тесты контрактов.

Как MDA увязывается с качеством и стандартами курса

Курс настаивает на **воспроизводимости**: стандарты, документация, автоматическая фиксация действий, «рельсы» процессов — всё это повышает предсказуемость результата. Эти принципы вы уже видели в лекции (контроль качества, отсутствие «непроверенных путей», ведение журналов и восстановимость «point-in-time»).

Что считать хорошей PIM и хорошей PSM

Хорошая PIM:

- Не содержит платформенных терминов (SQL-типы, REST-маршруты) — только предметная логика и поведение.
- Ясные границы и инварианты (вплоть до OCL).
- Покрывает ключевые сценарии, отразимые одной диаграммой последовательностей на сценарий.

Хорошая PSM:

- Полные контракты (OpenAPI/AsyncAPI/Proto) и схема данных под целевую СУБД.
- Видно, где обеспечиваются качества: доступность/восстановимость/латентность (SLO/SLI).
- Понятная диаграмма развёртывания: узлы, границы отказов, распределение компонентов (этот тип диаграммы подробно применяется в следующей ПР-5 для клиент-серверных и распределённых систем).

Ограничения и риски MDA

- **Инструментальная зависимость.** Без автоматизации трансформаций MDA превращается в «рисование ради рисования». OMG не даёт готовых тулов — их нужно выбирать и настраивать.
- • **Гранулярность моделей.** Слишком «плоская» PIM не даст пользы при трансформации; слишком детальная — утонет в шуме.
- • **Эволюция.** При изменении домена вы обязаны **мигрировать модель**, а не только код; иначе потеряется трассируемость.